

LLD Practice Platform — Design Document

Problem Statement

Engineers need a feedback loop to practice Low-Level Design: - **Input:** a structured design (classes, responsibilities, relationships, explanation) - **Output:** explainable rubric-based feedback with scores, evidence, concerns, suggestions

This is fundamentally an **LLD/domain-design exercise**, not a distributed systems problem. The focus is on **clean modular architecture**, **extensibility**, and **testability**.

Architectural Decisions

1. Modular Monolith (not Microservices)

Decision: Single Flask application with distinct module boundaries.

Rationale: - 2-day scope — microservices add operational overhead with no benefit at this scale - Allows easy deployment (one process, one DB) - Domain boundaries enforced through module structure, not network calls - Scaling path: extract modules as needed when scope grows

Boundaries:

```
domain/      ← Pure business objects (no I/O)
repositories ← All DB access (no business logic)
services     ← Orchestration only (no direct DB access)
evaluators/  ← Evaluation strategy (no Flask/DB dependency)
api/         ← HTTP translation only (no business logic)
```

2. Evaluator Strategy Pattern

Decision: BaseEvaluator interface with concrete implementations.

```
class BaseEvaluator(ABC):
    @abstractmethod
    def evaluate(self, problem: Problem, submission: Submission) → Evaluation:
        ...

    @property
    @abstractmethod
    def evaluator_type(self) → str:
        ...
```

Implementations:	Evaluator	Used for	Is Fallback	----- ----- -----	AIEvaluator	
Production (OpenAI GPT-4o)	No	RuleBasedEvaluator	Fallback / offline mode	Yes		

MockEvaluator | Tests | Yes |

Why this is good LLD: - Open/Closed: add new evaluators without changing service code - Dependency Inversion: service depends on `BaseEvaluator` interface - Factory pattern in `evaluator_factory.py` handles instantiation - All three are interchangeable; tests swap evaluators trivially

3. Repository Pattern

Decision: Repositories abstract all database access.

```
class ProblemRepository:
    def get_all(self) → List[Problem]:
    def get_by_id(self, id: str) → Optional[Problem]:
    def get_by_slug(self, slug: str) → Optional[Problem]:
```

Rationale: - Service layer never imports SQLAlchemy models - Domain entities are pure dataclasses — testable without DB - Mapping function `_map_problem(model) → Problem` is the only translation point - Repository can be swapped for in-memory impl in tests (currently uses SQLite)

4. Domain Entities as Dataclasses

Decision: Domain objects (`Problem`, `Attempt`, `Submission`, `Evaluation`, `RubricCriterion`) are pure Python dataclasses with no ORM dependency.

Rationale: - Entities can be constructed and tested without a DB session - Evaluators receive domain objects, not ORM models - Clearly separates "what the business cares about" from "how it's stored"

5. Background Evaluation Threading

Decision: `threading.Thread` for async evaluation with Flask app context captured before the thread starts.

```
# On request thread (has app context):
flask_app = current_app._get_current_object()
thread = Thread(target=self._run_evaluation, args=(flask_app, attempt_id, eval_id))
thread.start()

# Background thread (uses captured app, not proxy):
def _run_evaluation(self, flask_app, attempt_id, eval_id):
    with flask_app.app_context():
        ...
```

Key: `current_app` is a thread-local proxy. The actual `app` instance must be captured on the request thread and passed explicitly.

Trade-off: Simple but not production-grade. For scale, use Celery + Redis.

Submission always persisted first: If evaluation crashes, the learner's submission is safe in the DB and can be retried via `POST /api/attempt/:id/evaluation/retry`.

6. Pydantic for AI Output Validation

Decision: `AIEvaluationResponse` Pydantic model validates OpenAI JSON output.

```
class AICriterionResponse(BaseModel):
    name: str
    score: float = Field(ge=0, le=10)
    evidence: str = ""
    concern: str = ""
    suggestion: str = ""
    confidence: str = "medium"

class AIEvaluationResponse(BaseModel):
    criteria: List[AICriterionResponse]
    overall_score: float = Field(ge=0, le=100)
    summary: str = ""
    strengths: List[str] = Field(default_factory=list)
    improvements: List[str] = Field(default_factory=list)
```

Rationale: - Pydantic validation is used to validate and constrain structured AI output - Score bounds (0–10 per criterion, 0–100 overall) are enforced via `Field(ge=0, le=10)` - If validation fails, `AIEvaluator` catches the `ValidationError` and returns an `Evaluation(status='failed')`, which the learner can retry

7. Duplicate Submission Protection

Decision: Duplicate submission check before status validation.

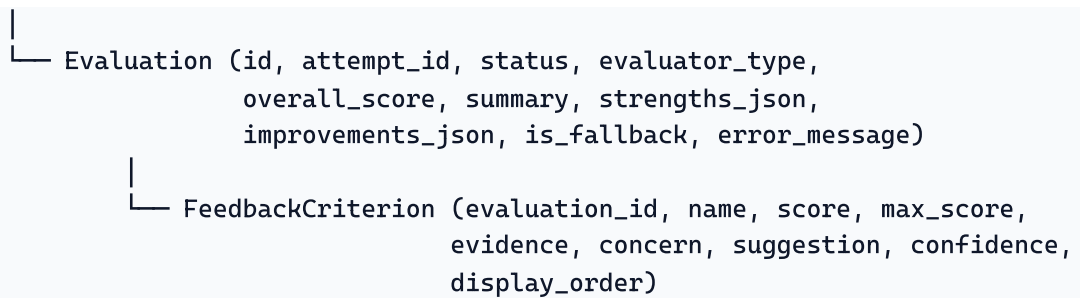
```
class DuplicateSubmissionError(ValueError): pass

existing = self._submissions.get_by_attempt_id(attempt_id)
if existing:
    raise DuplicateSubmissionError("A submission already exists for this attempt.")
```

This check is intentionally before the attempt status check — so it returns HTTP 409 (Conflict) rather than the more confusing 422 (status change) error on a duplicate. This is an application-level guard, not a database-level atomic operation.

Data Model

```
Problem (id, slug, title, difficulty, short_description,
         detailed_requirements, assumptions, submission_guidance)
├── Attempt (id, problem_id, learner_id, attempt_number, status)
│   ├── Submission (id, attempt_id, assumptions, classes_json,
│                   interfaces, relationships, design_explanation, edge_cases)
```



Rubric (8 Criteria)

#	Criterion	What it measures
1	Requirement Understanding	Are key features addressed?
2	Class Responsibilities	SRP adherence, focus
3	Encapsulation & Interfaces	Interfaces, abstractions, data hiding
4	Coupling & Cohesion	Dependency direction, cohesion
5	Abstraction / Design Patterns	Pattern usage, OOP principles
6	Extensibility	OCP, pluggability
7	Edge Cases & Testability	Failure modes, testable design
8	Design Explanation	Rationale, trade-off awareness

Each criterion is scored 0–10. Overall score = weighted average × 10.

Evaluation Flow (Happy Path)

```
POST /api/attempts/:id/submissions
├─ Validate request (Pydantic)
├─ Check for duplicate submission (→ 409 if exists)
├─ Validate content (must have class or explanation)
├─ Create Submission record
├─ Update Attempt → 'submitted'
├─ Create Evaluation record (status: 'pending')
├─ Capture flask_app reference
├─ Start background thread
│   └─ mark Evaluation 'evaluating'
│       └─ Update Attempt → 'evaluating'
│           └─ Call evaluator.evaluate(problem, submission)
```

```

├─ If result.status == 'failed':
│   └─ save failed result
│       └─ Update Attempt → 'failed'
└─ Else:
    └─ save completed result + criteria
        └─ Update Attempt → 'completed'

```

Evaluation Flow (Fallback)

```

AIEvaluator.evaluate()
├─ Call OpenAI API
├─ Parse JSON response
├─ Validate with Pydantic (AIEvaluationResponse)
├─ SUCCESS:
│   └─ Return Evaluation(status='completed', is_fallback=False)
└─ ANY FAILURE (timeout / API error / rate limit / bad JSON / Pydantic validation):
    └─ _rule_based_fallback()
        └─ RuleBasedEvaluator.evaluate() ← SUCCESS
            └─ Return Evaluation(status='completed', is_fallback=True)
        └─ RuleBasedEvaluator also fails
            └─ Return Evaluation(status='failed')
                └─ Service marks Attempt → 'failed'
                    └─ Learner can POST /evaluation/retry

```

Factory-level fallback (startup only):

```

EVALUATOR_TYPE=ai, but OPENAI_API_KEY missing or AIEvaluator import fails
└─ Factory substitutes RuleBasedEvaluator transparently (is_fallback=True)

```

Note: `MockEvaluator` is used exclusively in the test suite (`TestingConfig`). It is not part of any production fallback chain.

Evaluation State Lifecycle

The `Attempt` status and `Evaluation` status track progress independently.

Normal flow (AI success):

```

Attempt:    in_progress → submitted → evaluating → completed
Evaluation:                pending    → evaluating → completed (is_fallback=False)

```

Normal flow (AI fails → RuleBased fallback):

```
Attempt:    in_progress → submitted → evaluating → completed
Evaluation:                pending    → evaluating → completed (is_fallback=True)
```



Failure flow (AI fails AND RuleBased fails):

```
Attempt:    in_progress → submitted → evaluating → failed
Evaluation:                pending    → evaluating → failed
                                                    |
                                                    POST /evaluation/retry
                                                    |
Attempt:                evaluating → completed
Evaluation:            evaluating → completed
```

Key invariant: The `Submission` is always persisted to the database **before** the background evaluation thread starts. If evaluation fails for any reason, the learner's submission is safe and a retry is possible.

States defined in `entities.py` : - `AttemptStatus` : `in_progress` , `submitted` , `evaluating` , `completed` , `failed` - `EvaluationStatus` : `pending` , `evaluating` , `completed` , `failed`

Frontend Architecture

```
React Router (SPA)
├─ / → DashboardPage (stats, recent attempts, how-it-works)
├─ /problems → ProblemsPage (grid of problem cards)
├─ /problems/:id → ProblemDetailPage (requirements, start attempt)
├─ /attempt/:id → WorkspacePage (structured editor, submit)
├─ /attempt/:id/feedback → FeedbackPage (scores, criteria, retry)
└─ /history → HistoryPage (grouped by problem, improvement tracking)
```

Key design choices: - `services/api.ts` : single file for all API calls, error extraction, proxy config - `types/index.ts` : TypeScript types mirroring backend domain entities - Components: `Badge` , `Button` , `Card` , `Navbar` , `UI` (`ScoreRing`, `PageLayout`, etc.) - `FeedbackPage` polls every 3s while evaluation is in `pending` / `evaluating` state - `WorkspacePage` redirects to feedback if attempt is already submitted

Structured Text Submission — MVP Decision

Decision: Accept structured JSON with named fields rather than free-form text, code files, or diagrams.

Learners provide their LLD design as a structured payload covering: - `assumptions` — constraints the design is built around - `classes` — name, responsibility, methods, and notes for each class - `interfaces` — abstract interfaces or contracts defined - `relationships` — how classes relate

(inheritance, composition, dependency) - `design_explanation` — overall rationale and trade-offs - `edge_cases` — failure modes and boundary conditions handled

Why structured input: Structured input makes evaluation more consistent and allows both the `AIEvaluator` and the `RuleBasedEvaluator` to operate against the same representation. The `Submission.to_text_summary()` method serialises this structure into a readable prompt for the AI. The rule-based evaluator can directly inspect class counts, keyword presence, and field completeness.

Trade-off: Less expressive than actual code or a UML diagram, but significantly simpler and more reliable for a 2-day MVP. Future extensibility for code or diagram submission formats is listed under Future Improvements and is designed into the `Submission` dataclass (`submission_type` field).

Trade-offs

Decision	Trade-off	Mitigation
SQLite in dev	Not production-grade	<code>DATABASE_URL</code> env var accepts PostgreSQL
Thread-based eval	No retry on crash	<code>POST /evaluation/retry</code> endpoint
Rule-based fallback	Not semantic	Users see <code>is_fallback=True</code> in UI
No auth	Shared demo data	<code>learner_id</code> field ready for auth integration
Pydantic validation for AI output	Rejects malformed AI responses	Falls back to <code>RuleBasedEvaluator</code> ; if fallback also fails, evaluation is marked <code>failed</code> and retry is available
Background thread	Loses app context	Capture <code>app._get_current_object()</code> pre-thread
Structured text submission	Less expressive than code/diagrams	Simpler, consistent for MVP; extensible via <code>submission_type</code>

Scaling Path

1. **Database:** SQLite → PostgreSQL (just change `DATABASE_URL`)
2. **Evaluation:** Thread → Celery + Redis (extract `_run_evaluation` to Celery task)
3. **Authentication:** Add JWT middleware, scope attempts to users
4. **API:** Single Flask app → separate service if traffic grows
5. **AI:** Add rate limiting, caching, model fallback chain (4o → 3.5-turbo)